Introduction to machine-learning using scikit-learn

QLSC612

May 2026

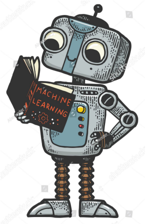
By

Nikhil Bhagwat & Mohammad Torabi

McGill UNIVERSITY

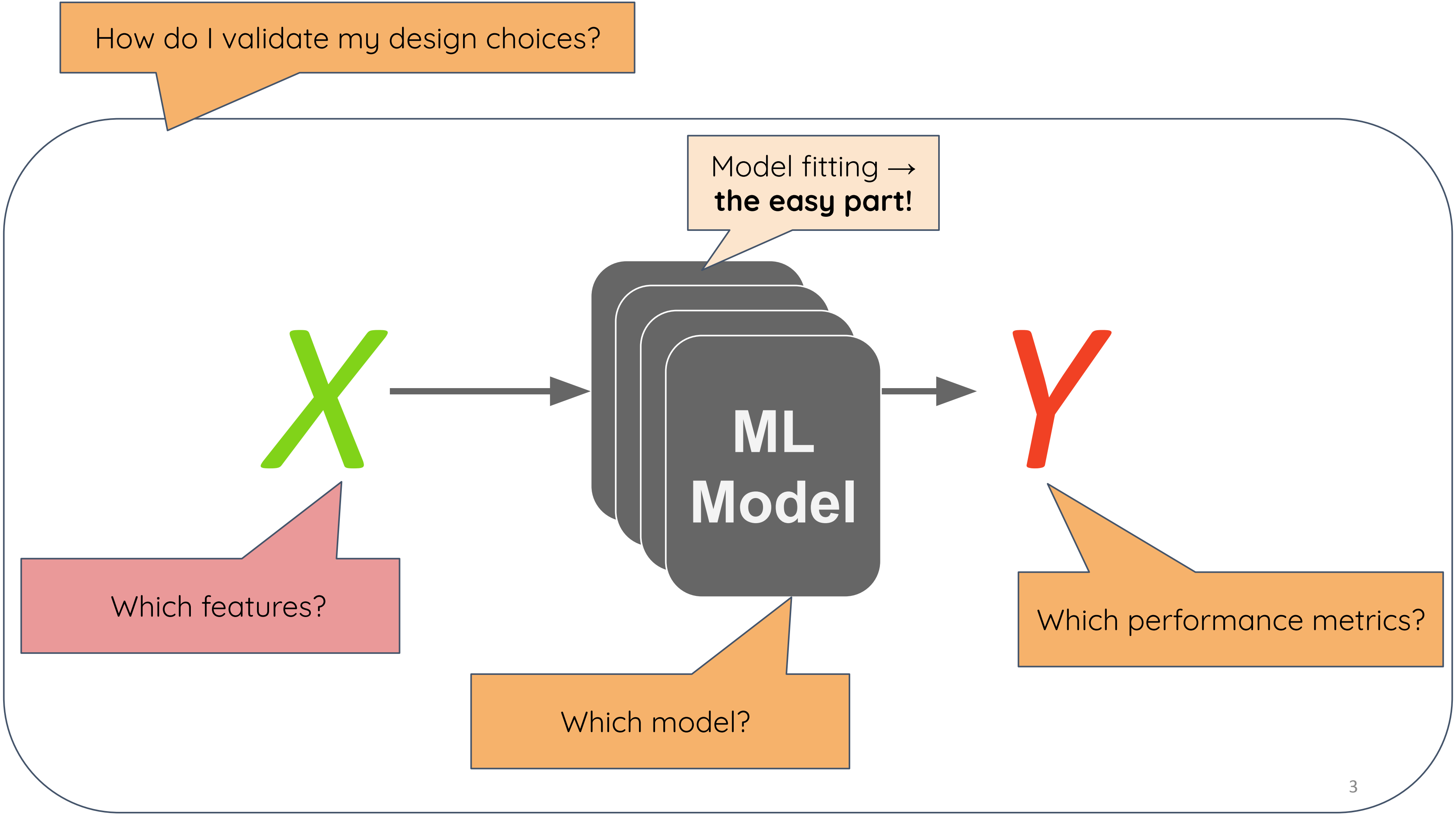
ORIGAMI Lab

neuro Montreal Neurological Institute-Hospital



Objectives

- Part 2: Unsupervised learning
  - **Dimensionality reduction**
  - Clustering
  - *Coding example: PCA, k-means*
  - *Coding example: fMRI site prediction*

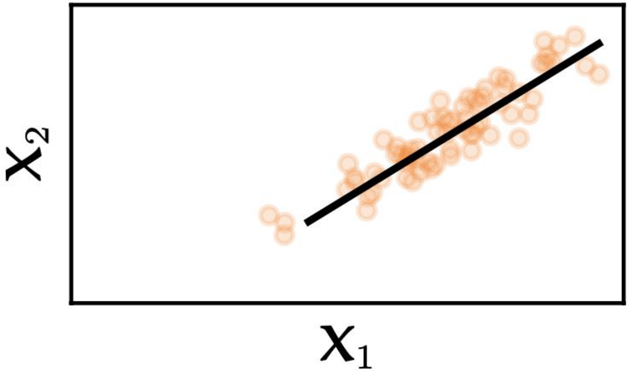

How do I validate my design choices?
Model fitting → **the easy part!**
X
ML Model
Y
Which features?
Which model?
Which performance metrics?

Unsupervised learning

- Learning without knowing the true labels
- Objectives
  - Dimensionality reduction of features
- Techniques
  - Feature Transformation/Projection

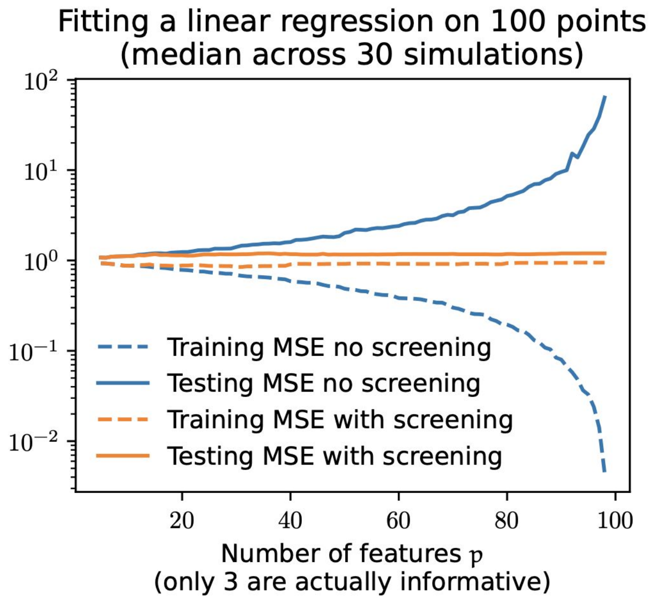
Dimensionality Reduction

Data is almost 1-dimensional
BUT represented as
2-dimensional



Dimensionality Reduction

- High dimensional data:
  - Model complexity increases -> unstable solution
  - Risk of overfitting: fitting exactly training data but failing on test data

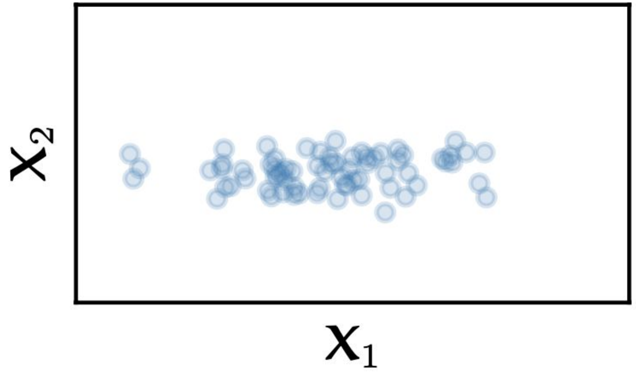


Dimensionality Reduction

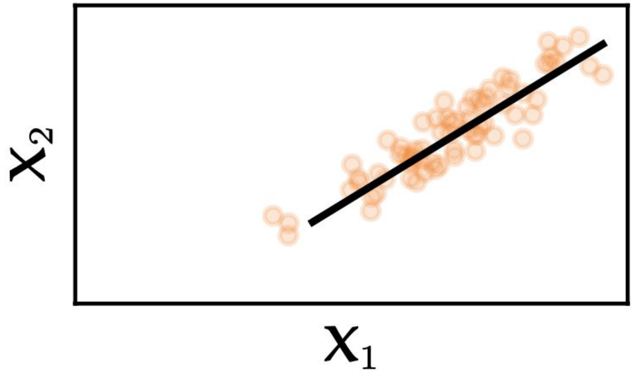
- **Curse of Dimensionality:** large number of input features can dramatically impact the performance of ML algorithms
- Techniques:
  - Feature selection (usually supervised)
  - Feature transformation (usually unsupervised)

Feature Transformation vs. Feature Selection

Maybe OK to drop X2

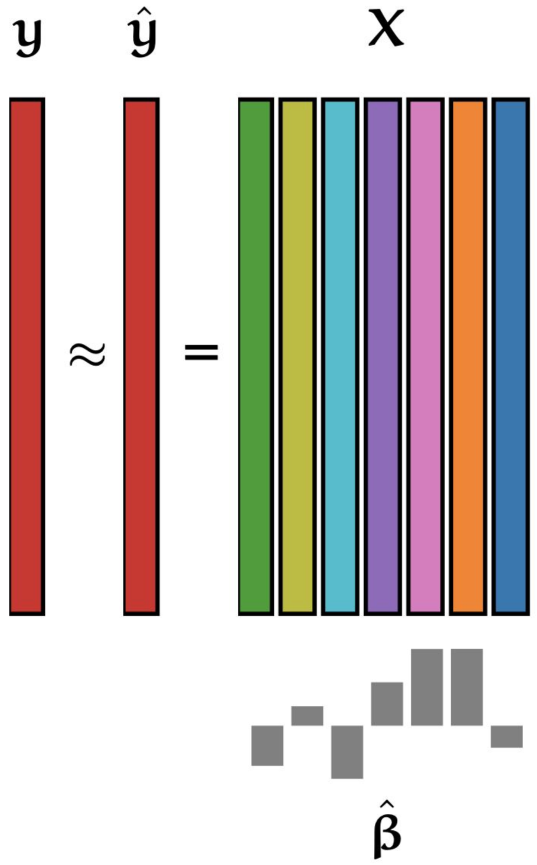


Data is low-dimensional BUT
no feature can be dropped



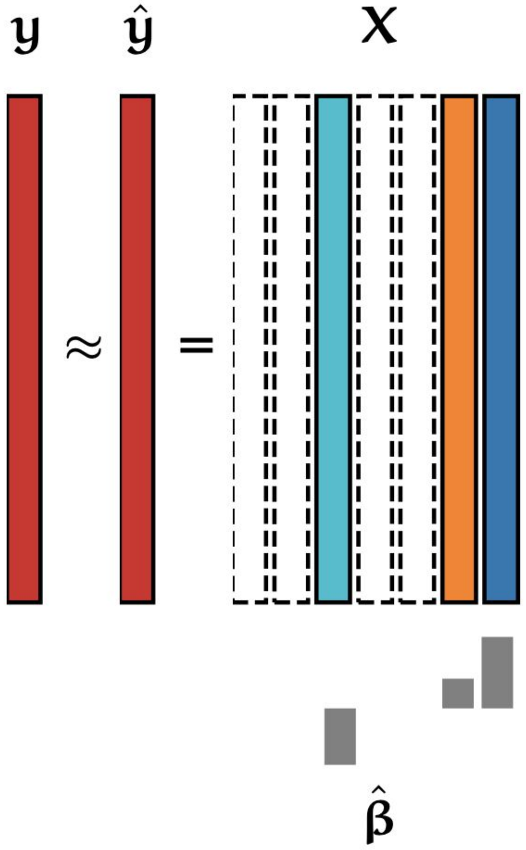
Feature Selection

$$\hat{\mathbf{y}} = \mathbf{X}\,\hat{\boldsymbol{\beta}}$$

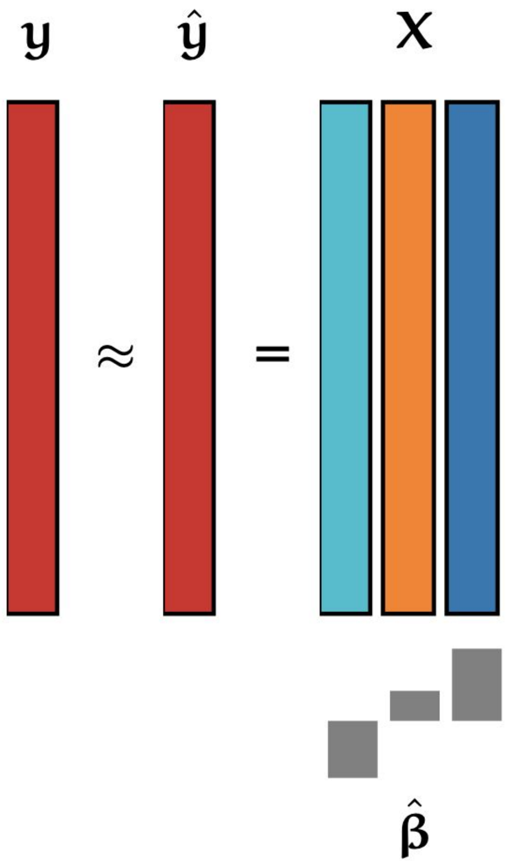


Feature Selection

$$\hat{\mathbf{y}} = \mathbf{X}\,\hat{\boldsymbol{\beta}}$$

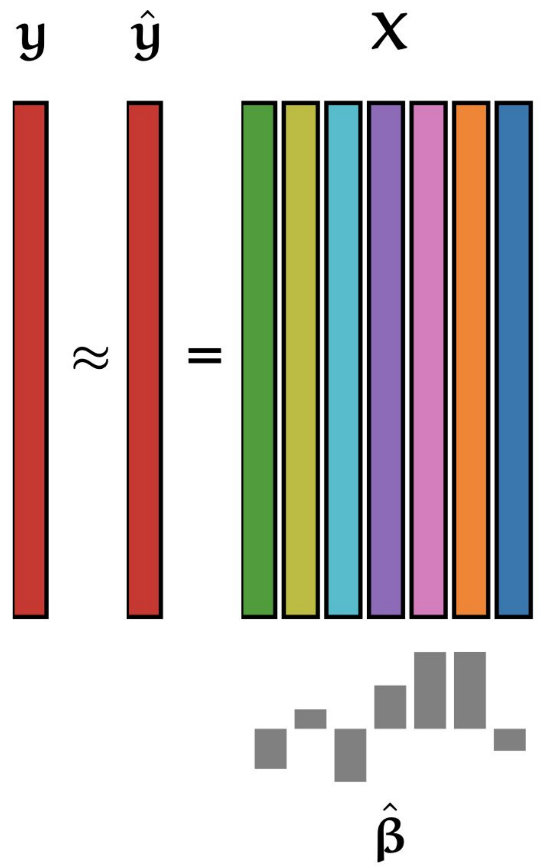


Feature Selection



Feature Transformation

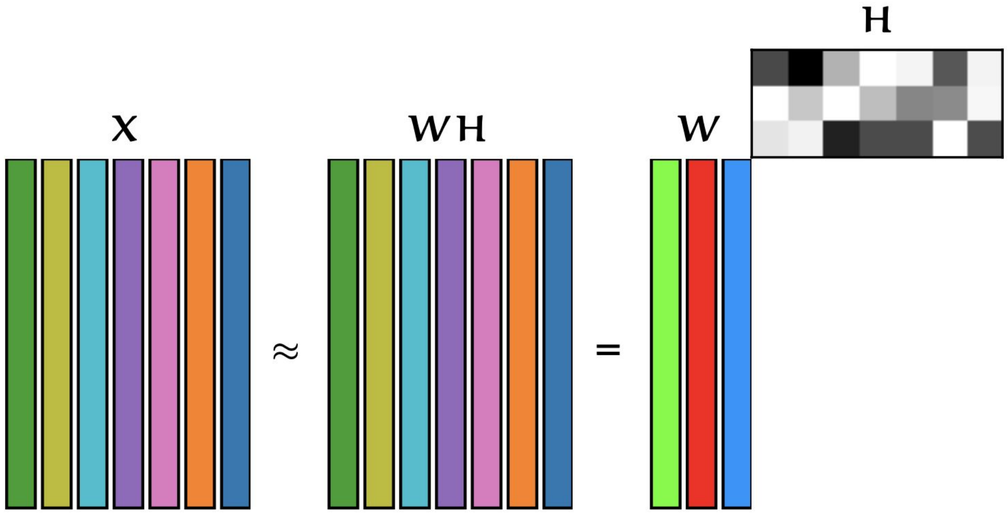
$$\hat{\mathbf{y}} = \mathbf{X}\,\hat{\boldsymbol{\beta}}$$



Feature Transformation

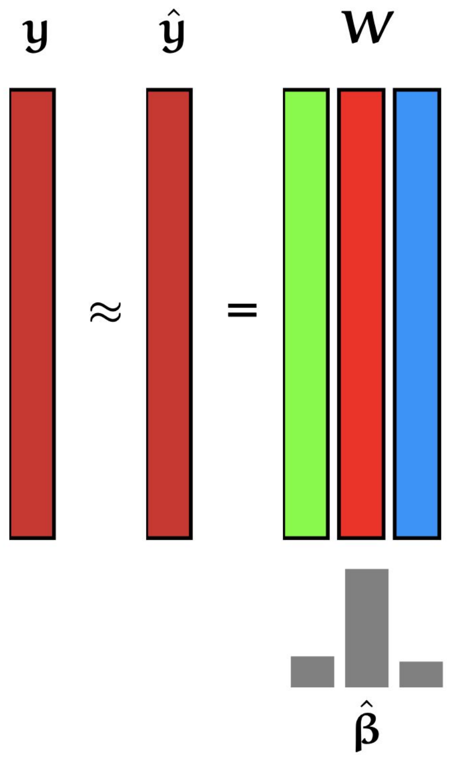
Minimize

$$\|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2 = \sum_{i,j} (\mathbf{X}_{i,j} - (\mathbf{W}\mathbf{H})_{i,j})^2$$



Feature Transformation

$$\hat{\mathbf{y}} = \boldsymbol{W}\hat{\boldsymbol{\beta}}$$



Feature Transformation

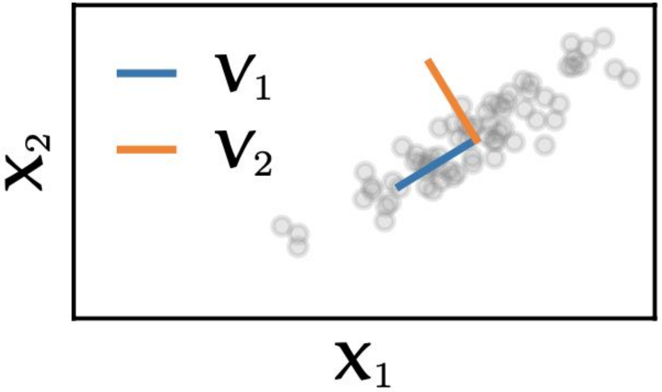
- Feature transformation is useful for
  - Information compression
  - Data artifact clean-up
  - Visualization

Feature Transformation

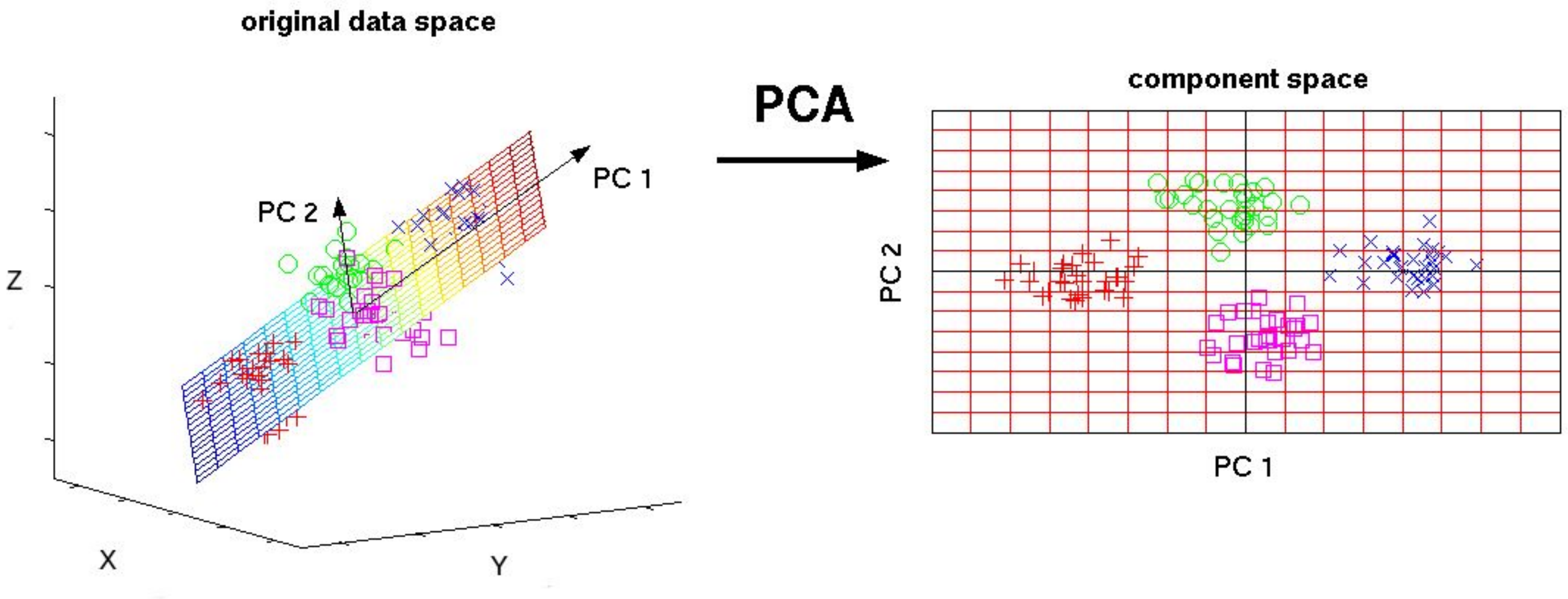
- Change of basis (i.e. “perspective”) for data representation
- Techniques with different priors on data generation process:
    - Singular Value Decomposition (SVD)
    - Principal Component Analysis (PCA)
    - Independent Component Analysis (ICA)
    - Non-negative Matrix Factorization (NMF)

Principal Component Analysis (PCA)

- PCA finds a set of components (eigenvectors) that are orthogonal and capture the maximum variance in data.
- These components form the basis of the new space that the data will be transformed to
- Truncating the components to keep only the first k components gives the best rank-k approximation of X and transforms X to k-dim space

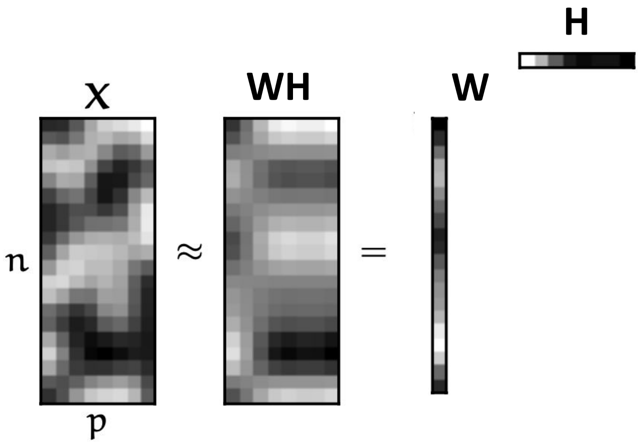


Principal Component Analysis (PCA)



Principal Component Analysis (PCA)

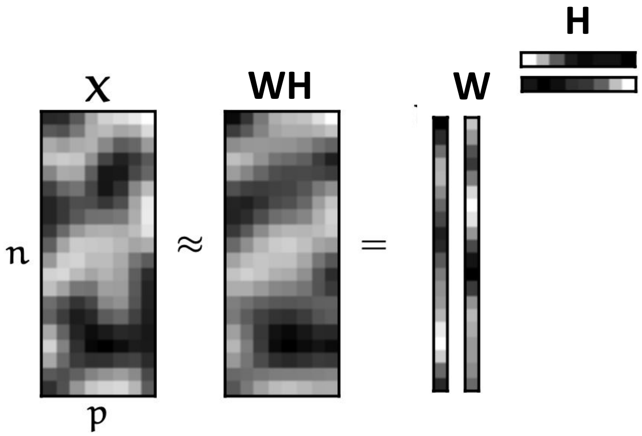
Reconstructing with 1 principal component:



Explained variance: 0.53

Principal Component Analysis (PCA)

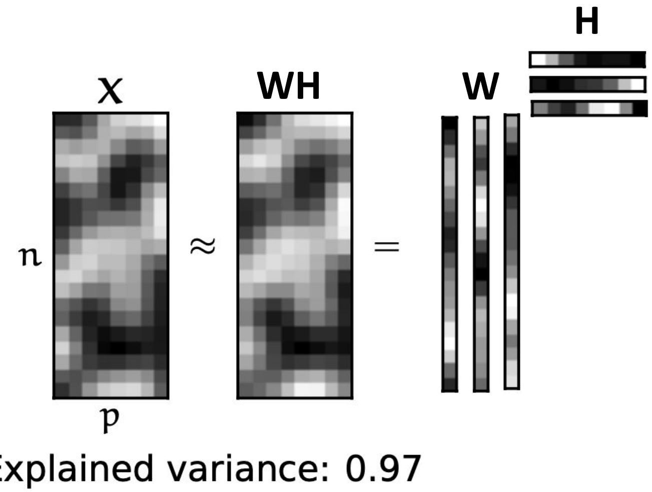
Reconstructing with 2 principal components:



Explained variance: 0.84

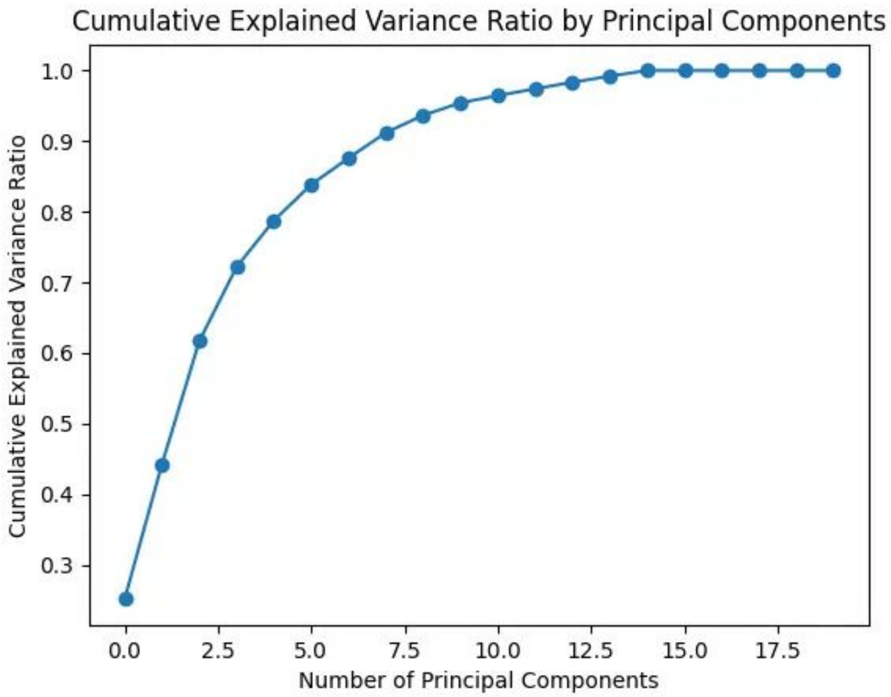
Principal Component Analysis (PCA)

Reconstructing with 3 principal components:

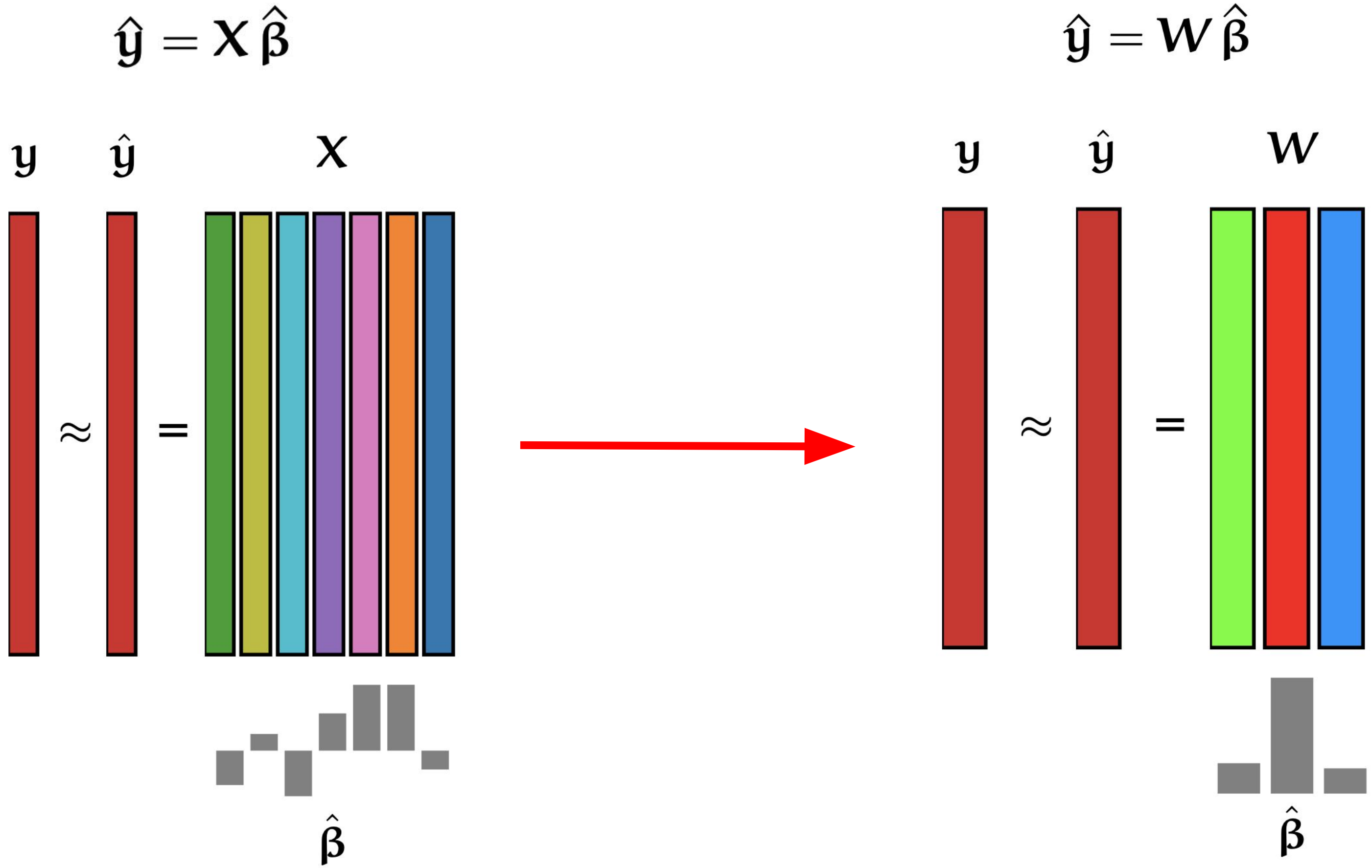


Principal Component Analysis

- Variance explained



Principal Component Analysis (PCA)



Principal Component Analysis

- sklearn

```
transformer = PCA(n_components=N)
transformer.fit(X)
transformed_X = transformer.transform(X)
print(transformed_X.shape) #(n_samples, n_components)
```

```
transformer = PCA(n_components=N)
transformed_X = transformer.fit_transform(X)
```

sklearn.decomposition.PCA